

vSmart Visibility AI Assistant 施工計畫

RAG-Doc Auto-Ingest + Shared Password Auth + Brief-Routing + Chunk
Fallback

v2.0 → v2.1 重大變化

- ✓ Auth 從 multi-user 簡化為 single shared password 「demo」
- ✓ 移除 SQLAlchemy User table、Argon2、admin/user role 區分
- ✓ 移除 create_user.py / reset_password.py CLI
- ✓ 所有人都是 admin (能 force ingest)
- ✓ OMP_Statistics_施工計畫_v0.2.pdf 確認進 corpus
- ✓ 專案目的擴展為三合一：內部工具 + portfolio + Agent 前哨
- ✓ 新增 Phase 11：Portfolio / Agent Infrastructure Polish

版本	v2.1 (locked)
日期	2026-05-02
Owner	Ming (Cisco UX / CNUX)
協作者	Claude Code (實作)
狀態	可動工 — 剩 Q6 待最後 confirm

目錄

1. v2.1 拍板狀態（密碼 demo + 三合一目的）
2. 架構簡化說明（multi-user → shared password）
3. Q6 解釋：OMP 施工計畫進 corpus 是什麼
4. 專案目標
5. Repo 現狀與最終結構
6. 三合一目的對應的設計取舍
7. 功能範圍
 1. Auth（簡化版）
 2. AI Assistant UI
 3. RAG-Doc Auto Ingestion
 4. VTT Parser
 5. PDF Parser + VLM
8. Brief-routing + Chunk fallback
9. LLM / Model 設定（不寫死 model ID）
10. Embedding 策略（multilingual）
11. Backend API 規格
12. Database Schema（簡化版）
13. Frontend 實作規格
14. 環境變數（v2.1 簡化）
15. 實作階段（11 phase）
16. Prompt 與回答政策
17. 風險與處理方式
18. Definition of Done（21 條）
19. Phase 11：Portfolio / Agent 化
20. 啟動方式
21. 給 Coding AI 的最終 prompt

1. v2.1 拍板狀態

1.1 你回覆的決策

Q	主題	你的回答
Q1	登入 gate 範圍	整 app gate 密碼 = demo (大小寫都可)
Q2	OAuth / SSO	不做
Q3	Admin CLI 建立 user	不要 → 等於說「不需要 user 系統」
Q4	Chat 歷史 UX	只 New Chat
Q6	OMP 施工計畫進 corpus	⚠ 待最後 confirm (建議放，理由見 § 3)
方向	專案真正目的	(a)+(b)+(c)+(d) 全部都是

1.2 三合一目的

(a) 內部工具給 Cisco 同事用

密碼 demo，公司內網丟連結就能用；Cisco PM/Eng 查 vSmart 過往會議與設計決策的工具。

(b) Portfolio piece — Anthropic / Cursor / Linear interviewer 看的

展示 Production-grade RAG + Auto-ingest + VLM + Brief-routing + SSE streaming + auth gate 的全棧能力。對應你的四個 skill gap：RAG / Multi-agent orchestration / LLM evals / Production reliability。

(c) Design Reasoning Agent 的前哨架構

這個 RAG infrastructure 之後可直接複製到 Design Reasoning Agent — 把 RAG-Doc 換成 50 個設計專案檔，Brief router 換成「PRD → 哪個專案經驗最相關」的判斷器。本系統等於 Design Reasoning Agent 的 v0 骨架。

(d) 同時三者都成立

內部工具是「能跑」的證明；portfolio 是「能講」的展示；Agent 前哨是「能複用」的槓桿。三合一意味著 Phase 11 必須加上 GitHub repo polish + Loom demo + 架構文件抽象化。

2. 架構簡化說明

2.1 為什麼從 multi-user 降級為 shared password 是對的

v2.0 原本設計了完整的多帳號系統：SQLite users table、Argon2 password hashing、admin/user role、CLI 工具、conversation 按 user_id 隔離。

你選 密碼 = demo，這是一個重要的 product decision —— 不是技術降級，是**正確的產品決策**。理由：

- **內部工具場景**：Cisco 同事不需要記另一個帳號；shared password 等於「公司內部演示密碼」的習慣
- **Portfolio 場景**：interviewer 要 demo 你的 app，給他密碼 5 秒進去比叫他註冊好十倍
- **Agent 前哨場景**：未來複製到 Design Reasoning Agent 時，auth 邏輯應該越輕越好
- **工時 ROI**：省 1.5–2 小時的 Phase 1 工作量，可以挪去 Phase 11 portfolio polish

移除的東西

原計畫	v2.1 處理
SQLAlchemy User model	✗ 移除
Argon2 password hashing	✗ 移除（單密碼比對即可）
admin / user role 區分	✗ 移除（所有人都是 admin）
<code>create_user.py</code> CLI	✗ 移除
<code>reset_password.py</code> CLI	✗ 移除（改密碼直接改 .env）
users table migration	✗ 移除
Phase 1 (Login / DB / User seed)	↔ 變成 Phase 1' (Single password gate)

保留的東西

- **JWT cookie**：仍要，這樣登入狀態能持久（不然每次刷新都要重打密碼）
- **SQLite + Alembic**：仍要，存 conversations / messages / file_hashes
- **Conversation persistence**：仍要，但欄位改為 `session_id`（不是 user_id）
- **HttpOnly + SameSite=Lax cookie**：仍要，保留基本安全
- **整 app gate**：仍要，未登入看不到 dashboard

2.2 簡化後的 Auth flow

1. 使用者開 `http://...:5173`
2. AuthProvider 呼叫 `GET /api/auth/me`
3. 沒 cookie 或 cookie 過期 → 401
4. 顯示 LoginScreen, 只有一個 password input field
5. 使用者輸入「demo」(或「DEMO」「Demo」都行 – case insensitive)
6. `POST /api/auth/login { password: "demo" }`
7. Backend 比對 `.env` 的 `APP_PASSWORD`
8. 通過 → set HttpOnly cookie, 內容是 `JWT { authenticated: true, session_id: uuid, exp: 7d }`
9. 前端進入 dashboard + AI Assistant
10. 之後 7 天內刷新都不用重打密碼

case insensitive 實作：backend 比對時 `password.lower() == app_password.lower()`，這樣 "demo" / "DEMO" / "Demo" / "DeMo" 都通過。

3. Q6 解釋：OMP 施工計畫進 corpus 是什麼

3.1 名詞解釋

Corpus（語料庫）= AI Assistant 能搜尋與回答的**知識庫文件範圍**。

這個 AI Assistant 的工作流程是：

- 1. 使用者問問題
- 2. 系統去 corpus 裡找相關文件
- 3. 把找到的文件丟給 LLM 當參考
- 4. LLM 根據參考文件回答

所以 **corpus 範圍 = AI 的世界範圍**。沒在 corpus 裡的東西，AI 永遠不會知道。

3.2 你的 RAG-Doc 目前有什麼

類型	數量	內容
會議 VTT + PDF 配對	9 對	Kickoff / PRD review / vSmart questions / Working session / EC Review / Post-EC follow-up 等
vSmart 產品 wiki PDF	2 份	vSmart insights on vManage - SDWAN - IT Wiki.pdf 與 retired 版
OMP 施工計畫.pdf	1 份	就是上一輪我幫你做的那份 25 頁 4MB PDF

3.3 兩種選擇

選 A：放進 corpus（建議）

未來能問：

- "Brad 為什麼質疑沒有 real-time？"
- "OMP 05:00 的 CPU 數值是多少？"
- "為什麼選 brief-routing 不用 chunk-RAG？"
- "v0.1 跟 v0.2 差在哪？"

AI 能答，並引用 OMP_Statistics_施工計畫_v0.2.pdf
• page 12。

選 B：不放

AI 只懂會議 + wiki，不懂你個人的施工計畫。

之後你正式定案、不再迭代了，再放進去。

缺點：你迭代到 v0.5 的中間，AI 沒辦法幫你查歷史決策。

3.4 為什麼建議選 A

- 1. **對應三合一目的：**施工計畫本身就是「Ming 的設計思考過程」展示材料 — 對 portfolio 與 Agent 訓練資料都有價值
- 2. **多版本不衝突：**brief 裡有 date 與 version 欄位，AI 可以區分新舊（甚至能答「v0.1 跟 v0.2 差在哪」）
- 3. **Auto-ingest 真實壓力測試：**你之後改 v0.3 拖進去，剛好驗證系統能正確處理檔案變更
- 4. **Agent 前哨的種子資料：**未來複製到 Design Reasoning Agent 時，這份施工計畫就是「Ming 的設計判斷模式」訓練樣本

v2.1 暫定採 A，但留給你最後確認的機會。 若你選 B，請在動工前明說，我修成 v2.2 並排除這份 PDF。

4. 專案目標

在既有 `vSmart-Visibility-main` 前端專案中新增一個 AI Assistant，固定顯示在頁面右下角。使用者輸入密碼 `demo` 後，可以透過聊天視窗詢問：

- meeting transcripts 內容
- VTT 逐字稿內容
- PDF 產品文件內容
- PDF 裡面的圖表、架構圖、表格
- `RAG-Doc/` 裡未來新增的文件

系統需要支援：FastAPI 後端 / 後端 `.env` 存 API key / RAG-Doc 自動 ingestion / VTT + PDF 解析 / VLM 處理圖像頁 / brief-routing 主路徑 / chunk-RAG fallback / SSE streaming / citation / 單密碼 gate / single-conversation 持久化。

5. Repo 現狀與最終結構

5.1 現狀

```
vSmart-Visibility-main/
├─ src/
│  └─ main.tsx
│    └─ app/
│       └─ App.tsx          # 5,786 行 Figma-converted layout
├─ vite.config.ts
└─ RAG-Doc/                 # 13 檔案 (會議 + wiki + 施工計畫)
```

5.2 最終結構 (v2.1 簡化)


```

vSmart-Visibility-main/
├── backend/
│   ├── pyproject.toml
│   ├── .env.example      # APP_PASSWORD=demo, ANTHROPIC_API_KEY=, model IDs...
│   ├── .gitignore
│   ├── README.md
│   └── data/
│       ├── corpus/      # 每檔提取後的 .txt
│       ├── chunks/      # 每檔的 .parquet
│       ├── briefs.json
│       ├── file_hashes.json
│       ├── failures.json
│       ├── brief_index.faiss
│       ├── chunk_index.faiss
│       └── app.sqlite    # 只存 conversations + messages
├── app/
│   ├── main.py
│   ├── config.py
│   └── api/
│       ├── auth.py      # POST /login, /logout, GET /me
│       ├── chat.py      # POST /chat (SSE)
│       ├── ingest.py    # POST /ingest, GET /ingest/status
│       └── sources.py    # GET /sources/{id}
│   ├── auth/
│       ├── security.py  # JWT encode/decode + password compare
│       └── deps.py      # get_session() - 取代 get_current_user()
│   ├── db/
│       ├── base.py
│       ├── session.py
│       ├── models.py    # 只有 Conversation + Message (無 User)
│       └── migrations/
│   ├── ingest/
│       ├── pipeline.py
│       ├── watcher.py
│       ├── worker.py
│       ├── store.py
│       ├── status.py
│       ├── chunker.py
│       ├── brief.py
│       ├── embed.py
│       └── parsers/
│           ├── vtt.py
│           ├── pdf.py
│           └── vlm.py
│   ├── rag/
│       ├── router.py
│       ├── retriever.py
│       ├── index.py
│       └── prompts.py
│   └── llm/
│       └── client.py
│   # ✗ 不再有 scripts/ 資料夾
│   # ✗ 不再有 auth/models.py / schemas.py / service.py
└── src/
    ├── app/
    │   ├── auth/
    │   │   ├── AuthProvider.tsx
    │   │   ├── LoginScreen.tsx  # 只有一個 password 欄位
    │   │   ├── useAuth.ts
    │   │   └── api.ts
    │   ├── components/
    │   │   └── ai-assistant/

```

```
├── AIAssistant.tsx
├── FAB.tsx
├── ChatPanel.tsx
├── Message.tsx
├── Citation.tsx
├── IngestStatusFooter.tsx
├── useChat.ts
├── useIngestStatus.ts
├── api.ts
├── types.ts
└── App.tsx

├── vite.config.ts
└── RAG-Doc/
```

6. 三合一目的對應的設計取舍

因為這專案要同時滿足三個目的，幾個設計決定必須兼顧：

主題	內部工具	Portfolio	Agent 前哨	v2.1 取舍
Auth	越輕越好	能 demo 就好	不應綁死	shared password （三方共贏）
Code 抽象度	能跑就好	要可讀	要可複用	類別介面 + 可替換 (embed/router/llm 都用 interface)
Repo 可見性	private OK	必須 public	private OK	public (GitHub) + README 寫清楚 + Loom demo
README 寫給誰	同事	interviewer	未來的自己	interviewer 第一 （含 architecture diagram + design decisions log）
Model ID 寫法	固定即可	專業度	必須可換	全進 .env （已是 v2.1 規範）
是否 Docker	同事友善	專業度	方便複製	Phase 11 加 docker-compose
Citation 細緻度	能信	展示精緻度	未來複用	file + page/timestamp + clickable
VLM 圖表處理	nice-to-have	必殺技	可複用	必做 + 在 README 強調
Streaming	UX 好	看起來厲害	—	SSE 必做
Brief routing 演算法	—	核心賣點	核心賣點	必須在 README 寫成設計決策文章

7. 功能範圍

7.1 Auth (v2.1 簡化版)

目標

整個 app 需要先輸入密碼 `demo` 才能進入。未登入者看到 LoginScreen，不應看到 dashboard 或 AI Assistant。

v2.1 預設決策

項目	設定
Login gate	整個 app
Auth method	Single shared password
密碼	<code>demo</code> (在 <code>.env</code> 裡可改)
大小寫	不敏感 (DEMO / Demo / demo 都通過)
Session	JWT in HttpOnly + SameSite=Lax cookie
Session duration	7-day rolling
User store	無 (不需要)
Conversation 隔離	按 <code>session_id</code> (瀏覽器 cookie 內)
Roles	無區分 (所有人都能 force ingest)

7.2 AI Assistant UI

UI 狀態

```
Closed:
- 只顯示 FAB button (右下角)
- icon: Sparkles 或 Bot

Open:
- slide-up chat panel
- message list
- input textarea
- send button
- citation chips
- ingest status footer
```

互動

- 點 FAB 開啟 / 點 X 關閉
- Enter 送出 / Shift+Enter 換行 / Esc 關閉
- Cmd+/ 開啟 assistant
- 送出後立即顯示 user bubble
- assistant 用 SSE streaming 逐字顯示
- 回答完成後顯示 citation chips
- "New Chat" 按鈕清空當前 conversation

Empty state 文字

標題：Ask questions about meetings, vSmart visibility, OMP statistics, or documents in RAG-Doc.

3 個 suggested questions：

- What was decided about OMP statistics?
- Summarize the latest meeting.
- What did the team say about routes and peers?

7.3 RAG-Doc Auto Ingestion

四種 ingestion trigger（必須全做）

1. Startup scan

後端啟動時掃描 RAG-Doc 內所有檔案

2. File watcher

watchdog 監聽 add / modify / delete / rename

3. Periodic safety scan

每 5 分鐘 hash-check 一次

4. Manual endpoint

POST /api/ingest?force=true

Debounce

拖入 PDF 時可能收到多次 modify event。等檔案大小穩定 2 秒後才 enqueue。

預設：`INGEST_DEBOUNCE_MS=2000`

Hash cache

- 每個檔案 SHA-256 hash 存 `file_hashes.json`
- 未變更 → skip
- 變更 → re-parse / re-brief / re-chunk / re-embed / update index

Delete / Rename

- **Delete**：移除 brief / corpus / chunks / vector entries / hash record
- **Rename（hash 相同）**：只更新 filename metadata，不重做 VLM/embedding

支援檔案類型

階段	副檔名
v1 必做	<code>.vtt</code> <code>.pdf</code>
v1 可選	<code>.txt</code> <code>.md</code>
v2 再做	<code>.docx</code> <code>.pptx</code> <code>.xlsx</code>

7.4 VTT Parser

```
{
  file: string,
  speaker: string,
  start_ts: string,
  end_ts: string,
  text: string
}
```

Chunking 規則：

- 依 speaker turns 合併
- 約 600 tokens 一個 chunk
- 保留 1 turn overlap
- metadata 必含 file / speaker list / start_ts / end_ts

Citation 顯示：`filename.vtt · 00:13:24–00:15:02`

7.5 PDF Parser + VLM

VLM trigger 條件（節省成本）

以下情況才把 page 送 VLM：

1. page image area > 30%
2. page text < 100 words
3. page 有 chart / graph / diagram / table image

VLM 輸出內容

- 圖表標題、x-axis、y-axis、legend、數值趨勢
- UI 區塊、表格欄位
- 架構圖節點與關係
- 重要設計決策

每頁存成

```
[PAGE 12]
Direct text:
...

Visual description:
...

Metadata:
file=...
page=12
```

Citation 顯示：`filename.pdf · page 12`

8. Brief-routing + Chunk fallback

8.1 Ingest 時永遠建立五層資料

```
1. extracted full text    → data/corpus/<file>.txt
2. brief                 → briefs.json
3. brief embedding       → brief_index.faiss
4. chunks               → data/chunks/<file>.parquet
5. chunk embedding       → chunk_index.faiss
```

8.2 Brief schema

```
{
  "file_id": "stable-id",
  "filename": "example.pdf",
  "path": "RAG-Doc/example.pdf",
  "type": "pdf",
  "hash": "sha256...",
  "title": "Document title",
  "date": "2026-05-02",
  "version": "v0.2",           // 給 OMP 施工計畫用
  "participants": ["Name A"],
  "topics": ["OMP statistics", "routes", ...],
  "gist_one_liner": "One sentence summary.",
  "summary": "Longer structured summary.",
  "created_at": "...",
  "updated_at": "..."
}
```

8.3 Query routing strategy (隨檔案數量階梯)

```
<= 50 files: router 直接看全部 briefs
50-200 files: 先用 brief_index 找 top 30, router 只看 top 30
> 200 files: 同上, 永遠不讓 router prompt 無限增長
```

8.4 Router 輸入 / 輸出

輸入：user question + last 6 conversation turns + candidate briefs

輸出 (必須 JSON)：

```
{
  "matches": [
    {
      "file_id": "file-001",
      "confidence": 0.84,
      "reason": "Question mentions OMP statistics and event chart behavior."
    }
  ]
}
```

8.5 Confidence gate

```
if max(confidence) >= 0.5:  
    使用 file-level answer # 載入 top 3-5 個檔案全文送給 LLM  
else:  
    使用 chunk-RAG fallback # 從 chunk_index 找 top 12 chunks
```


9. LLM / Model 設定

🔥 不要把 model name 寫死在程式碼裡。全部進 .env。

實作時必須用 web search 或 docs.claude.com 確認當下可用的 model ID，不要沿用舊文件裡的字串。

```
ANTHROPIC_API_KEY=  
ANTHROPIC_CHAT_MODEL=  
ANTHROPIC_ROUTER_MODEL=  
ANTHROPIC_VISION_MODEL=  
ANTHROPIC_BRIEF_MODEL=
```

用途	需求
CHAT_MODEL	最終回答 / SSE streaming / 品質高
ROUTER_MODEL	brief table routing / JSON output / 快與便宜
VISION_MODEL	PDF chart 描述 / ingest 階段
BRIEF_MODEL	每檔結構化 brief 生成

10. Embedding 策略

文件含繁體中文（OMP 施工計畫.pdf 通篇中文）+ 英文（會議 VTT 與 wiki）→ 必須用 multilingual embedding model，不可用純英文模型。

```
Embedding model: multilingual embedding model  
Vector store: FAISS
```

實作時要設計成可替換介面：

```
class Embedder:  
    def encode(self, texts: list[str]) -> np.ndarray:  
        ...
```

不要把 embedding 模型綁死在 retrieval logic 裡。

11. Backend API 規格

11.1 Health

```
GET /api/health  
→ { "status": "ok" }
```

11.2 Auth

Login

```
POST /api/auth/login  
{ "password": "demo" }  
  
Response:  
{ "ok": true }  
(同時 set HttpOnly cookie 包含 JWT { authenticated: true, session_id, exp })
```

Logout

```
POST /api/auth/logout  
(清除 cookie)
```

Current session

```
GET /api/auth/me  
→ { "authenticated": true, "session_id": "uuid" }  
失敗 → 401
```

11.3 Chat (需登入)

```
POST /api/chat  
{  
  "conversation_id": "optional-uuid",  
  "messages": [  
    { "role": "user", "content": "What did the team decide about OMP statistics?" }  
  ]  
}
```

Response 為 SSE stream，event types：`text` / `citation` / `done` / `error`

```
{ "type": "text", "content": "The team decided..." }

{
  "type": "citation",
  "citation": {
    "file": "meeting.vtt",
    "page": null,
    "timestamp": "00:12:10-00:13:40",
    "label": "meeting.vtt · 00:12:10"
  }
}

{ "type": "done", "conversation_id": "uuid" }
```

11.4 Ingest status

```
GET /api/ingest/status
→ {
  "indexed": 13,
  "processing": ["new-meeting.pdf"],
  "queued": ["new-meeting.vtt"],
  "failed": [
    { "file": "corrupt.pdf", "reason": "PyMuPDF invalid xref", "attempts": 3 }
  ],
  "last_scan": "2026-05-03T14:22:11Z"
}
```

11.5 Manual ingest

```
POST /api/ingest?force=true
(v2.1: 所有登入 session 都可呼叫, 因為沒有 admin/user 區分)
```

11.6 Source file opening

```
GET /api/sources/{source_id}
```

安全要求

- 必須檢查 session 已登入
- 只能讀 `RAG_DOC_DIR` 內檔案
- 防止 path traversal (拒絕 `../../secret.env`)
- 用 `file_id` 而非 raw path 查找

12. Database Schema (v2.1 簡化版)

v2.1 移除 users table。所有 conversation 按 cookie 裡的 `session_id` 隔離。

conversations

```
id          UUID PRIMARY KEY
session_id  UUID NOT NULL    -- 來自 cookie，不是 user_id
title       TEXT
created_at  TIMESTAMP
updated_at  TIMESTAMP
```

messages

```
id          UUID PRIMARY KEY
conversation_id UUID FK
role        TEXT    -- user / assistant
content     TEXT
citations_json TEXT
created_at  TIMESTAMP
```

(optional) ingest_files

v1 先用 `file_hashes.json`；v2 再考慮搬到 DB。

13. Frontend 實作規格

13.1 App.tsx

```
<AuthProvider>
  <AppShell />
</AuthProvider>

未登入 → <LoginScreen />
已登入 → <> <ExistingDashboard /> <AIAssistant /> </>
```

13.2 LoginScreen 設計

- 置中 card
- 標題：vSmart Visibility
- 副標：Enter password to continue
- 單一 password input field
- Submit button
- 錯誤訊息（密碼錯時顯示）
- 不要顯示 hint 「password is demo」 — 但你可以告訴你的同事

13.3 AuthProvider 負責

- app 啟動時呼叫 `/api/auth/me`
- 保存 session 狀態
- `login(password)` / `logout()`
- loading state / auth error state

13.4 useChat 負責

- 發送 `POST /api/chat`
- 讀取 SSE stream

- append streaming text
- append citations
- done 後保存 `conversation_id`
- error handling

13.5 IngestStatusFooter

Chat panel 開啟時每 5 秒 polling `GET /api/ingest/status` 。顯示：

```
13 docs indexed
2 indexing...
1 failed
```

Indexing 中也允許提問，但提示：

Some documents are still indexing. Answers may not include newly added files yet.

14. 環境變數

backend/.env.example :

```
# — Auth —————
APP_PASSWORD=demo
APP_PASSWORD_CASE_INSENSITIVE=true
JWT_SECRET_KEY=change-me-to-a-random-string
JWT_EXPIRE_DAYS=7

# — Anthropic —————
ANTHROPIC_API_KEY=

# Model IDs - 實作時用 web search 確認當下可用 ID
ANTHROPIC_CHAT_MODEL=
ANTHROPIC_ROUTER_MODEL=
ANTHROPIC_VISION_MODEL=
ANTHROPIC_BRIEF_MODEL=

# — RAG —————
RAG_DOC_DIR=/home/julia/002_AT/Ming/vSmart-Visibility-main/RAG-Doc

ROUTER_CONFIDENCE_THRESHOLD=0.5
ROUTER_MAX_FILES=5
ROUTER_CANDIDATE_BRIEFS=30
CHUNK_FALLBACK_TOP_K=12

# — Ingest —————
INGEST_CONCURRENCY=2
INGEST_DEBOUNCE_MS=2000
INGEST_RESCAN_INTERVAL_S=300
INGEST_WATCH_RECURSIVE=false

# — DB —————
DATABASE_URL=sqlite+aiosqlite:///./data/app.sqlite
```

15. 實作階段（11 phase）

Phase	名稱	v2.1 變動
0	Backend scaffolding	—
1'	Single password gate + DB	大簡化：無 User table，password compare + JWT cookie
2	Frontend auth gate	LoginScreen 只有單 password 欄位
3	Document parsers	—
4	Brief generation	新增 <code>version</code> 欄位（給 OMP 施工計畫）
5	Chunking + Embeddings + FAISS	multilingual embedding
6	Auto-ingest pipeline	—
7	Retrieval + Answering	—
8	Chat API + Conversation persistence	scoped by <code>session_id</code> 不是 <code>user_id</code>
9	Frontend AI Assistant UI	—
10	Security / QA / Polish	—
11	Portfolio / Agent Polish（新）	見 § 19

Phase 1' 簡化後的具體工作

```
# backend/app/auth/security.py
def verify_password(input_password: str) -> bool:
    expected = settings.APP_PASSWORD
    if settings.APP_PASSWORD_CASE_INSENSITIVE:
        return input_password.lower() == expected.lower()
    return input_password == expected

def create_session_jwt() -> str:
    payload = {
        "authed": True,
        "session_id": str(uuid.uuid4()),
        "exp": now() + timedelta(days=settings.JWT_EXPIRE_DAYS)
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY)

# backend/app/auth/deps.py
async def get_session(request: Request):
    cookie = request.cookies.get("session")
    if not cookie:
        raise HTTPException(401)
    try:
        payload = jwt.decode(cookie, settings.JWT_SECRET_KEY)
        return SessionContext(session_id=payload["session_id"])
    except (JWTError, ExpiredSignatureError):
        raise HTTPException(401)
```

沒有 SQLAlchemy User model、沒有 Argon2、沒有 admin role check。整個 phase 從 ~1.5 hrs 降到 ~30 min。

各 phase 驗收條件（節錄關鍵點）

- Phase 0： `curl /api/health` → 200

- **Phase 1'** : POST `{password: "demo"}` 成功 set cookie ; `{password: "DEMO"}` 也通過 ; `{password: "wrong"}` 401
- **Phase 2** : 未登入只看到 LoginScreen ; 登入後看到 dashboard ; refresh 後仍登入 ; logout 後回 LoginScreen
- **Phase 3** : 13 個檔案都有 `data/corpus/<file>.txt` ; OMP PDF chart 頁有 visual description
- **Phase 4** : `briefs.json` 有 13 筆 ; OMP 施工計畫的 brief 含 `version: "v0.2"`
- **Phase 5** : 問中文「OMP 路由」與英文「OMP routes」都能找到相關 chunks
- **Phase 6** : drag 新 VTT 進 RAG-Doc → 10 秒內 indexed ; rename 不重 VLM ; delete 同步移除
- **Phase 7** : 問「Brad 為什麼質疑 real-time」能答並引用 OMP 施工計畫
- **Phase 8** : refresh 後當前 conversation 保留 ; 不同瀏覽器 (不同 session_id) 看不到彼此 conversation
- **Phase 9** : FAB 在右下 ; 點開 panel 能 streaming 回答 ; citation chip 可點開原檔
- **Phase 10** : API key 不在前端 bundle ; source endpoint 拒絕 path traversal
- **Phase 11** : 見 § 19

16. Prompt 與回答政策

16.1 Answer system prompt

```
You are an AI assistant for vSmart Visibility.  
Answer only using the provided retrieved documents.  
Cite every factual claim with source file and timestamp/page.  
If the answer is not in the documents, say you could not find enough evidence.  
Follow the user's language.  
For meeting transcripts, preserve who said what when available.  
For PDFs, cite page numbers.  
For visual descriptions, say it is based on the extracted visual description.  
For documents with version metadata (e.g. v0.1, v0.2),  
mention the version when answering, and note if multiple versions exist.
```

16.2 Router prompt 必須輸出 JSON

```
{  
  "matches": [  
    { "file_id": "...", "confidence": 0.0, "reason": "..." }  
  ]  
}
```

17. 風險與處理方式

風險	處理方式
PDF 圖表頁很多，VLM 成本高	只處理 image-heavy 或 text-sparse pages，concurrency cap 2
文件持續增加，brief table 太大	超過 50 files 後先用 brief embedding 找 top 30
router 選錯檔案	confidence gate < 0.5 走 chunk fallback
使用者問新檔案但還沒 indexed	UI 顯示 indexing status，回答提示資料可能尚未完整
Anthropic rate limit	concurrency 預設 2 + exponential backoff
embedding 首次下載慢	startup warm load，README 註明
PDF parser 失敗	單檔 failure，不阻塞 queue
檔案複製中被 ingestion	debounce + file size stable check
source file 被亂讀	限 RAG_DOC_DIR + path traversal 防護
model ID 過期	全進 .env，實作前確認可用 model
密碼洩漏（v2.1 新風險）	只用於內部 demo；勿放 public 雲服務；如要 deploy 公開請換 multi-user 架構
OMP 施工計畫多版本混淆	brief 含 <code>version</code> ，answer prompt 規範要 mention 版本

18. Definition of Done (21 條)

1. 未輸入正確密碼時不能看到 dashboard
2. 密碼 `demo` 大小寫都通過
3. refresh / login / logout 流程正常
4. 右下角 AI assistant 可開啟與關閉
5. 問問題後可 SSE streaming 回答
6. 回答會顯示 citation
7. citation 可指向 PDF page 或 VTT timestamp
8. citation chip 可點擊開原檔
9. RAG-Doc 新增檔案後會自動 ingestion
10. PDF 中圖表 / UI screenshot 會被 VLM 轉成文字描述
11. startup scan 可補捉 server 關閉期間新增的檔案
12. delete / rename 會同步更新 index
13. `/api/ingest/status` 能顯示 indexed / queued / processing / failed
14. API key 不出現在 frontend bundle
15. 不同 session (不同瀏覽器) 的 conversation 分離
16. router confidence 低時會 fallback 到 chunk-RAG
17. 問不存在資料時不 hallucinate
18. 中英文問題都能照使用者語言回答
19. OMP 施工計畫被正確分版本回答
20. README 有完整啟動步驟 (含 Loom demo 連結)
21. Phase 11 portfolio polish 全部完成 (見 § 19)

19. Phase 11 : Portfolio / Agent Infrastructure Polish

本 phase 對應「三合一目的」中的 (b) portfolio + (c) Agent 前哨。完成 Phase 0-10 後再做。

19.1 GitHub repo polish

- repo 設為 public (命名建議: `vsmart-visibility-ai`)
- 頂層 README 寫成 portfolio 級別:
 - 1 段「Why I built this」(對應你目標 \$500K role 的 narrative)
 - 1 段 architecture diagram (用 Mermaid 畫 brief-routing + chunk-fallback + auto-ingest)
 - 1 段「Design decisions log」(為什麼用 brief-routing 不用純 chunk-RAG / 為什麼 multilingual embedding / 為什麼 confidence gate)
 - 3 個 GIF (auto-ingest / streaming chat / citation click)
 - Loom demo 連結 (< 2 分鐘)
- License: MIT
- Issues 模板 (讓 interviewer 看到工程素養)

19.2 Loom demo 腳本 (< 2 分鐘)

1. 0:00-0:15 一句話定位: what + who for + why
2. 0:15-0:45 demo 1: 問會議內容 → streaming + citation
3. 0:45-1:15 demo 2: 問 PDF 圖表內容 → VLM-extracted 描述
4. 1:15-1:40 demo 3: drag 新檔進 RAG-Doc → 自動 indexed

5. 1:40–2:00 結語：「未來會抽象成 Design Reasoning Agent」

19.3 Architecture 抽象化 (Agent 前哨用)

- 把 RAG-Doc 路徑、brief schema、router prompt 都抽到 config layer
- 未來複製到 Design Reasoning Agent 時，只需換：
 - `RAG_DOC_DIR` → 50 個 design 專案資料夾
 - Brief prompt → 從「meeting/spec brief」改成「design project brief」
 - Router prompt → 從「找會議」改成「找最相關的設計判斷經驗」
 - Answer prompt → 從「客觀引用」改成「以 senior UX designer 第一人稱推理」
- Documentation：`docs/REUSE.md` 寫清楚怎麼複製到別的 domain

19.4 Docker compose

- `docker-compose.yml` 一鍵啟動 backend + frontend
- README 加「One-command demo」區段

19.5 Production readiness 加分項

- Health check endpoint 含詳細 dependency status (Anthropic / FAISS / FS)
- Structured logging (JSON format)
- Basic metrics endpoint (query latency / VLM call count / cache hit rate)
- Test：每個 phase 至少 1 個 integration test

19.6 Phase 11 驗收

- repo 設 public 且 README 達 portfolio 級別
- Loom demo < 2 分鐘上傳完成
- 3 個 GIF 嵌入 README
- `docker-compose up` 一鍵啟動
- `docs/REUSE.md` 寫清楚 Agent 化路徑
- Architecture diagram 用 Mermaid 嵌 README

20. 啟動方式

Backend

```
cd backend
cp .env.example .env
# 填入 ANTHROPIC_API_KEY
# APP_PASSWORD 預設 demo，要改也行
pip install -e .
uvicorn app.main:app --reload
```

Frontend

```
npm install
npm run dev
```

開啟 `http://localhost:5173`，密碼輸入 `demo`

Docker (Phase 11 後)

```
docker-compose up
```

21. 給 Coding AI 的最終 prompt

Please implement the vSmart Visibility AI Assistant per construction plan v2.1.

KEY CHANGES FROM v2.0:

1. AUTH IS SIMPLIFIED – no User table, no Argon2, no admin/user roles.
Single shared password from APP_PASSWORD env var.
Case-insensitive comparison (DEMO / Demo / demo all work).
Still use JWT HttpOnly cookie for session persistence (7-day rolling).
2. Conversations scoped by session_id (from cookie), not user_id.
3. Phase 1 is renamed Phase 1' – single-password gate, ~30 min not 1.5 hr.
4. NEW Phase 11 – Portfolio / Agent infrastructure polish.

CRITICAL CONSTRAINTS:

1. Do not expose ANTHROPIC_API_KEY to the browser.
2. Do not hardcode Anthropic model IDs – all model names must come from .env.
3. Verify current available Anthropic model IDs at implementation time using web search or docs.claude.com. Do not assume model IDs.
4. Use multilingual embedding model (corpus contains Chinese – OMP 施工計畫.pdf).
5. Source file endpoint must prevent path traversal.
6. VLM only on PDF pages with image area > 30% or text < 100 words.
7. Brief routing first, chunk-RAG fallback when confidence < 0.5.
8. SSE streaming for /api/chat.
9. Auto-ingest: startup scan + watchdog + periodic rescan + manual endpoint.
10. Implement phase by phase and verify each phase before moving on.
11. Brief schema must include "version" field (for OMP 施工計畫).
12. Answer prompt must mention version when documents have multiple versions.

PURPOSE (affects polish priorities in Phase 11):

- (a) Internal tool for Cisco colleagues
- (b) Portfolio piece for Anthropic / Cursor / Linear interviewers
- (c) Architectural prototype for future Design Reasoning Agent
- (d) All of the above – so make architecture cleanly reusable.

START WITH: Phase 0 – Backend scaffolding.

After each phase, verify the listed acceptance criteria and ask for confirmation.

22. 最後確認

v2.1 動工前剩一個問題

所有其他決策已 lock :

- ☒ Q1 整 app gate + 密碼 demo (case insensitive)
- ☒ Q2 不做 SSO
- ☒ Q3 不要 user 系統 → 改成 shared password
- ☒ Q4 只 New Chat 按鈕
- ☒ 三合一目的 → 加 Phase 11 polish
- ☒ Model ID 全進 .env 不寫死
- ☒ Embedding 走 multilingual

剩一個確認：

Q6 : OMP_Statistics_施工計畫_v0.2.pdf 進 corpus ?

- ☒ 進 (v2.1 採取, 因為對 portfolio + agent 前哨都有價值)

- **✗** 不進（如果你覺得太私人 / 怕亂入）

回覆「v2.1 進 corpus 確認」或「不進 corpus」即可動工。